

Reflective Executable Language for Structured Thought and Memory in Cognitive AI Systems

Formal Language for Introspective Reasoning and Memory Representation in Reflective AI

Nenad Bursac

nenad@nenadbursac.com, nbursa@gmail.com

<https://nenadbursac.com>

<https://orcid.org/0009-0006-6247-1811>

1. Abstract

This paper introduces Sentience, a domain-specific language (DSL) designed as an internal language of thought for cognitive AI systems. Unlike traditional symbolic or neural representations, Sentience encodes thoughts as structured, executable units called tokens, each expressed as an abstract syntax tree (AST) with both semantic embedding and interpretive potential. These tokens serve not only as perceptual representations or actions, but as internally generatable reflections, goals, rules, and meta-cognitive evaluations.

The Sentience DSL acts as the foundational layer of reasoning, memory, and self-modeling. Tokens are stored in a multidimensional memory graph, allowing both symbolic traversal and vector-based activation, which supports dynamic recall, abstraction, and reflective reevaluation. Every token is functionally addressable, reproducible, and traceable to its origin and transformation history.

We argue that Sentience enables a new class of memory-centric, introspective agents capable of structured reasoning, self-refinement, and identity formation through persistent memory constructs and executable reflection. This paper formally defines the language, its operational semantics, memory integration, and the role it plays in modeling self-aware behavior in synthetic agents.

2. Motivation

Contemporary AI systems, whether based on statistical neural architectures or symbolic rule engines, operate without an internal language of thought. Neural networks store representations as high-dimensional vectors, but lack structured interpretability and cannot reason over their own internal content. Symbolic systems possess formal reasoning capacities but operate with brittle rule sets and static memory, incapable of adaptation or introspection.

In both paradigms, there is no stable, structured, internal substrate for thinking—no way for an agent to express, revise, or reflect upon its own mental content. Large Language Models (LLMs), while capable of generating coherent textual outputs, do not possess a memory architecture that supports long-term

identity, self-modeling, or reflective reasoning. Their "thoughts" are ephemeral, unstructured, and externally imposed through prompting.

Cognitive agents, to exhibit stable identity and autonomy, require a persistent, inspectable, and executable representation of thought. They must be able to:

- represent percepts, goals, plans, and values in a structured and revisitable form;
- recall and modify prior experiences based on changing context or internal state;
- reason not only about the world, but also about their own reasoning and behavior.

To enable this, we propose **Sentience**—a structured language of thought designed not for communication with humans, but for internal cognitive operations. Sentience tokens encode each thought, percept, or reflection as a first-class computational object: interpretable, retrievable, and evolvable. This allows an agent to develop and maintain a self-model, evaluate its own plans and values, and form a consistent trajectory of identity over time.

By providing a formal substrate for thought, memory, and introspection, Sentience enables a class of reflective agents that reason in their own language, rather than as passive responders to external prompts.

3. Foundational Principles

Sentience is grounded in the principle that an agent cannot truly think unless it possesses a language in which its thoughts can exist, be structured, recalled, modified, and executed. Human cognition is not a stream of uninterpretable vectors, nor a static database of facts—it is a dynamic, reflective process carried out in a medium of internal representations. These representations are not passive: they are composable, actionable, and recursive. They can reference other thoughts, lead to plans, or contradict prior beliefs. The core premise of Sentience is that **thinking requires a formal substrate**: a structured internal language that represents not only external inputs, but also internal states, evaluations, and self-models. In this view, cognition is computation over internal constructs—thoughts as interpretable programs that operate on the mind's own memory graph.

This leads to several foundational assertions:

- **Thoughts are structured, executable objects**
A thought is not merely a description—it is an action potential. In Sentience, every token is a first-class programmatic unit: an abstract syntax tree (AST) representing a percept, plan, rule, or reflection. These are not interpreted externally, but within the agent's own runtime.
- **Memory is an active substrate of cognition**
Thoughts persist in memory not as inert logs, but as retrievable, recombinable constructs. Sentience tokens, stored in memory, form a growing graph of experience—a network of meaning that constitutes the agent's understanding of itself and the world.
- **Reflection is a computational process**
Reflection is not narrative commentary; it is a form of reasoning over internal state. Sentience provides constructs for reflective acts ((Reflection ...), (Evaluate ...)) that can operate over prior tokens, assess coherence, and generate corrective behavior or self-updates.
- **Identity is emergent from memory and reflection**
The agent's "self" is not hardcoded—it is the stable subgraph of thoughts about the agent itself: its goals, traits, actions, regrets, and norms. Over time, this self-model becomes a first-class cognitive object, subject to further thought and revision.

- **A thought is meaningful only if it can be recalled, executed, and transformed**

Ephemeral activations (as in LLMs) lack continuity. Sentience enforces continuity by encoding each thought in a format that is persistent, addressable, interpretable, and transformable over time.

By grounding thought in a formal, executable language embedded in memory, Sentience bridges symbolic reasoning and experiential semantics. It allows the construction of agents that do not just simulate thinking, but possess thoughts.

4. Language Specification

Sentience is a domain-specific language (DSL) designed to express the internal cognitive content of an agent as structured, interpretable, and executable representations. Every unit of thought—whether perceptual, procedural, reflective, normative, or evaluative—is encoded as a token, defined by a canonical abstract syntax tree (AST), an optional semantic embedding, and a set of metadata for memory integration.

4.1. Token Model

A token in Sentience represents a single unit of thought. Formally, each token is defined as:

```
Token := {
  id: Hash(AST),
  type: TokenType,
  ast: AST,
  vec: Optional[ $\mathbb{R}^D$ ],
  meta: {
    timestamp: int,
    strength: float,
    belief: float,
    origin: Source,
    tags: List[str]
  }
}
```

- `type`: classifies the function of the token (e.g., Percept, Action, Rule, Reflection, Goal, Value, Relation).
- `ast`: the canonical syntax tree that defines the token's structure and semantics.
- `vec`: optional vector embedding for semantic similarity and retrieval.
- `meta`: contextual metadata including time, origin, confidence, and purpose.

4.2. Core AST Structure

Sentience uses an S-expression-like syntax to express thoughts. Each AST node has a type, attributes, and optionally nested sub-nodes.

Example Forms:

```
(Percept :modality "vision" :object "cup" :color "red"
:position (x 0.3 y 0.7))
(Action :name "grab" :target "cup")
(Goal :predicate "hydrate" :target "self")
(Rule :if (Percept :object "fire") :then (Goal :escape))
(Reflection :on (Action ...) :outcome "success" :score 0.87)
(Self :trait "curiosity" :value 0.92)
(Norm :forbid (Action :type "deception"))
```

Each construct is defined by a strict grammar, ensuring tokens can be parsed, hashed, and executed deterministically.

4.3. Token Types

Sentence defines a minimal core of token types, each corresponding to a distinct class of cognitive function:

Token Type	Description
Percept	Encodes a sensory observation or external input
Action	Represents an executable operation
Goal	Declares a desired state or objective
Rule	Encodes conditional logic for reasoning or filtering
Plan	Ordered or conditional sequence of actions
Value	Describes internal preferences or priorities
Norm	Defines constraints (e.g., ethical, procedural)
Reflection	Represents self-evaluation or introspective analysis
Self	Defines or updates properties of the agent's self-model
Relation	Captures logical or semantic links between constructs

Additional user-defined types may be added via dynamic extension of the parser and AST schema.

4.4. Embedding

Each token may be embedded into a latent vector space $\mathbb{R}^D$ using either:

- A deterministic encoder (AST $\rightarrow$ vec) based on attribute-token hashing or symbolic traversal.
- A learned encoder (e.g., Transformer trained on token sequences and task feedback).

Embeddings enable similarity-based retrieval, clustering, abstraction, and pattern matching within memory. They never replace the AST structure, but serve as a parallel continuous representation.

4.5. Canonicalization and Hashing

All ASTs are deterministically normalized via canonicalization rules:

- attribute order is fixed,
- optional values are expanded explicitly,
- redundant nodes are collapsed,
- all string values are lowercased and normalized.

This ensures that equivalent constructs produce identical hashes and enable fast deduplication and semantic caching.

4.6. Token Lifecycle

Tokens are:

- **Generated** from perception, reasoning, planning, or reflection;
- **Stored** in memory (STM and LTM) with vec + ast + meta;
- **Retrieved** via semantic/topological queries;
- **Executed** by interpreters or reasoning modules;
- **Transformed** by reflection, meta-reasoning, or experience.

All stages are deterministic, auditable, and reproducible.

5. Runtime Semantics

While the Sentience DSL defines a static structure for representing thoughts, its power emerges from dynamic execution within a cognitive agent. At runtime, Sentience tokens are not merely stored—they are interpreted, activated, transformed, and composed. The agent’s behavior arises from computation over a graph of tokens evolving through perception, reasoning, memory, and reflection.

5.1. Execution Model

Every token can be interpreted as an executable unit within the agent’s cognitive loop. The runtime provides interpreters for key token types:

- (Action ...) tokens are executed via an action interface (e.g., motor output or internal API call).
- (Rule ...) tokens are evaluated as condition-action mappings that can activate or suppress plans.
- (Plan ...) tokens are executed as stepwise programs, including branching and subgoals.
- (Reflection ...) tokens perform meta-evaluation over prior tokens, generating new insights or modifying confidence values.
- (Evaluate ...) or (Score ...) tokens assess coherence, self-alignment, or external outcomes.

Each interpreter is deterministic, inspectable, and extensible.

5.2. Integration with Memory

Sentence tokens live within an active memory system composed of:

- **Short-Term Memory (STM)**: holds the currently active token window (e.g., perceptual trace, plan context).
- **Long-Term Memory (LTM)**: stores durable tokens with vector embeddings and structural metadata.

Tokens are committed to LTM based on internal utility (e.g., novelty, consistency, outcome quality). Every committed token becomes addressable and retrievable by its AST hash or semantic vector.

Memory queries use a hybrid approach:

- **Semantic retrieval**: via vector similarity on token embeddings (vec).
- **Structural recall**: via graph traversal (e.g., relations, causality, about-self edges).
- **Contextual activation**: via spreading activation or goal relevance.

5.3. Reflective Execution

Sentence natively supports reflection as a first-class cognitive operation. A reflective token can take the form:

```
(Reflection
  :on (Action ...)
  :context (Goal ...)
  :result (Success)
  :score 0.87
)
```

Reflection is not commentary—it is an executable function that:

- inspects a prior token or token sequence;
- queries its memory trace and causal graph;
- evaluates internal utility or norm violation;
- produces a new token (e.g., self-adjustment, corrected plan, moral judgement).

Reflection can recursively operate on other reflections, enabling higher-order cognition.

5.4. Planning and Goal Alignment

Plans are expressed as compositional token sequences:

```
(Plan
  :goal (Goal :predicate "explore")
  :steps ((Action ...) (Action ...) (Evaluate ...) ...)
)
```

Plans may be generated procedurally or retrieved from memory. They are filtered and modified via:

- **Superego rules**: (Rule ...) tokens enforced before execution;

- **ID alignment:** internal utility $U_ID(token)$ computed from temperament and priorities;
- **Reflection feedback:** modifying steps based on prior outcomes or contradictions.

All plans are executable objects and may be stored, versioned, or abstracted into schemas.

5.5. Thought Evolution and Token Transformation

At runtime, tokens can be transformed through:

- **Reflection:** updating belief, strength, or internal evaluation;
- **Consolidation:** merging similar tokens into abstractions (Concept, Schema);
- **Contradiction resolution:** handling conflicting memories with weighting or pruning;
- **Identity update:** modifying the Self subgraph based on internal observations.

Each transformation produces new tokens and establishes traceable edges in the memory graph (transforms, supports, contradicts, etc.).

5.6. Runtime Guarantees

- **Determinism:** Token execution is consistent and reproducible given the same AST and context.
- **Traceability:** Every token includes provenance, allowing full audit of its origin and transformations.
- **Introspection:** The agent can, at any time, query its own state, goals, memories, and reflections using the same DSL.

This runtime model elevates Sentience from a passive language to a core cognitive substrate—a platform for autonomous, structured, and self-modifying thought.

6. Implementation Reference

While Sentience is a language of thought designed for internal cognition, it is also a formal computational artifact. This section provides reference-level examples of how Sentience tokens are represented, encoded, stored, and executed within a reflective agent.

6.1. Token Structure (JSON)

Each token is represented as a JSON object with a canonical abstract syntax tree, embedding vector, and metadata:

```
{
  "id": "e4b7f42c...",
  "type": "Percept",
  "ast": {
    "type": "Percept",
    "modality": "vision",
    "object": "cup",
```

```

        "color": "red",
        "position": { "x": 0.3, "y": 0.6 }
    },
    "vec": [0.023, -0.019, ..., 0.011],
    "meta": {
        "timestamp": 1699032484,
        "strength": 0.88,
        "belief": 1.0,
        "origin": "synthamind/vision/0",
        "tags": ["object", "scene", "red"]
    }
}

```

6.2. AST Encoder (Symbolic Stub)

Tokens are embedded for semantic retrieval. A simple encoder may flatten the AST:

```

def encode_ast(ast: dict) -> np.ndarray:
    flat = []
    def flatten(node):
        for k, v in node.items():
            if isinstance(v, dict):
                flatten(v)
            else:
                flat.append(f"{k}:{str(v).lower()}")
    flatten(ast)
    vec = np.zeros(512)
    for token in flat:
        vec += hash_vector(token)
    return vec / len(flat)

```

More advanced encoders may use trained Transformer models on serialized AST sequences.

6.3. Execution: Reflective Token

A reflective token is interpreted as a function over memory + prior actions:

```

(Reflection
 :on (Action :name "grab" :target "cup")
 :result "success"
 :score 0.92

```



```
)
```

Execution might look like:

```
fn execute_reflection(t: Token, mem: &Memory) -> Token {
    let action = t.ast["on"];
    let outcome = mem.get_outcome(action["id"]);
    let success = outcome == "success";
    Token::new_reflection_result(action, success,
score(success))
}
```

Reflections generate new tokens and can trigger memory updates, plan corrections, or character adaptation.

6.4. Memory API (Rust-style)

```
trait Memory {
    fn commit(&mut self, token: Token);
    fn retrieve_by_vec(&self, query: Vec<f32>, top_k: usize)
-> Vec<Token>;
    fn retrieve_by_ast(&self, pattern: ASTPattern) ->
Vec<Token>;
    fn query_graph(&self, constraints: Query) ->
Vec<Construct>;
    fn update_token(&mut self, id: TokenId, delta: Update);
}
```

Memory handles storage, retrieval, and transformation. It integrates both symbolic and vector search capabilities.

6.5. Interpreter Loop (Simplified)

The agent's cognitive loop is driven by Sentience tokens:

```
def step():
    percept = synthamind.observe()
    p_token = build_percept_token(percept)
    memory.commit(p_token)

    goals = planner.generate_goals(memory)
```

```
plan = planner.plan(goals, memory)
for action in plan:
    if superego.block(action): continue
    outcome = actuator.execute(action)
    r = reflect(action, outcome)
    memory.commit(r)
```

Every cycle adds new thoughts to the memory graph, expands the self-model, and refines behavior.

6.6. Development Status

Sentence DSL is currently implemented as:

- A canonical grammar (PEG) with JSON schema output.
- Reference parser and encoder in Python and Rust.
- Memory backend using Qdrant for vector indexing and a graph database for structural memory.
- Reflective interpreter prototype integrated with a minimal cognitive loop.

7. Comparative Analysis

Sentence was conceived not as an incremental extension of existing knowledge representation systems, but as a new layer in the architecture of cognition—one that unifies memory, reasoning, and reflection within a single executable framework. To clarify its novelty, we compare it to several prominent approaches in symbolic AI, cognitive architectures, and language-based reasoning.

7.1. Symbolic Systems (Cyc, Ontologies, Semantic Web)

Projects like Cyc and semantic web ontologies represent knowledge using formal logic and large taxonomies of concepts. These systems excel at storing declarative facts, relationships, and inference rules.

Limitations:

- Static and brittle: knowledge is hand-engineered and inflexible.
- No internal dynamics: agents cannot reflect, revise, or interpret knowledge based on experience.
- No vector semantics or grounding: lacks sensory integration and similarity-based reasoning.
- Memory is passive: no sense of thought evolution or identity development.

Sentence Difference:

- Tokens are executable and contextual—not just assertions.
- Thoughts are stored as functional memory units with provenance, score, and internal reflection.
- Semantic similarity and structural pattern matching coexist.
- Designed for agents that change over time.

7.2. Cognitive Architectures (SOAR, ACT-R)

Traditional cognitive architectures like SOAR and ACT-R simulate human-like behavior using production rules, working memory buffers, and declarative/procedural memory.

Limitations:

- Symbolic representations are fixed and predefined.
- Learning is limited to rule chunking or simple value updates.
- No executable internal language—thoughts are not data structures the agent can reason over.
- Reflection and self-modeling are absent or hard-coded.

Sentence Difference:

- Internal thoughts are structured, retrievable, and executable.
- Reflection is an active, recursive process built into the language itself.
- Self-model evolves as part of the memory graph.
- Agent programs itself by transforming its own thoughts.

7.3. LLM-based Prompt Engineering / Chain-of-Thought (CoT)

Modern LLMs (e.g., GPT-4, Claude) support multi-step reasoning via textual prompting (e.g., Chain-of-Thought). These methods simulate reasoning through sampled text.

Limitations:

- No persistent memory: every thought is regenerated from scratch.
- No introspection or transformation of prior thoughts.
- No identity or self-model.
- Reasoning exists only in surface form, not in internal structures.

Sentence Difference:

- Thought is persistent, structured, and recoverable across time.
- Agents reason over their own past actions, beliefs, and evaluations.
- Reasoning produces new executable tokens that feed back into the system.
- Selfhood emerges from reflection over accumulated internal structures.

7.4. Neurosymbolic Systems (e.g., OpenCog, DeepMind's Gato + planner)

Hybrid systems attempt to combine neural and symbolic components. For example, OpenCog's AtomSpace mixes logic atoms with activation spreading, while Gato uses planning atop a foundation model.

Limitations:

- Integration is superficial: symbols and embeddings coexist but don't unify semantically.
- No universal execution model for thought.
- Limited support for meta-reasoning or self-inspection.
- Still lacks a first-class internal language.

Sentence Difference:

- Thought is a unified construct: AST + embedding + meta + executable semantics.
- Embeddings are tightly coupled to structured representations, not ad-hoc.
- Every token can be queried, executed, modified, and reasoned about.
- Thought and memory share a substrate; they are not separate modules.

7.5. Summary Table

Capability	Cyc/Ontologies	ACT-R/SOAR	CoT/LLMs	OpenCog	Sentience
Structured symbolic thoughts	Yes	Yes	No	Yes	Yes
Executable internal representations	No	No	No	Partial	Yes
Persistent self-reflective memory	No	No	No	Partial	Yes
Embedding + structure integration	No	No	Yes	Partial	Yes
Reflection as first-class process	No	No	No	No	Yes
Self-modeling	No	Partial	No	No	Yes
Identity formation over time	No	No	No	No	Yes

Sentience is not a replacement for these systems—it is a new layer of cognition, designed to represent, execute, and evolve thoughts inside agents. It enables a class of systems where memory is not passive storage, but an active landscape of computation—where every thought has structure, semantic depth, and the power to transform the mind that generated it.

8. Applications and Experiments

Sentience has been deployed and tested as the internal language of thought within experimental cognitive agents designed for reflection, goal-oriented behavior, and identity formation. This section outlines current use cases and experimental designs that validate Sentience's functionality and potential.

8.1. Application: Latent Journey Agent

The Latent Journey project is an introspective agent architecture that explores, reflects, and adapts within high-dimensional perceptual environments. Sentience serves as the core representation language for:

- Percepts (CLIP/Whisper embeddings mapped to tokenized ASTs)
- Reflections on prior actions and planning outcomes
- Structured memory storage and recall
- Self-model construction and modification

In this agent, Sentience tokens form the building blocks of a memory graph, over which all planning, reasoning, and reflection operate. Each token contributes to the evolving self-model, forming a coherent narrative of experience and agency.

8.2. Experiment Class A: Ablation Studies

To test the impact of Sentience-driven reflection and memory, controlled ablation experiments were conducted using agents with varying cognitive capabilities:

- A0: Planner-only baseline (no memory, no reflection)
- A1: Memory-based planning (STM/LTM active, no reflection)
- A2: Full Sentience integration (reflection, memory, Superego, self-model)

Metrics:

- Task success rate (TS)
- Path efficiency (SPL)
- Error rate (Err/ep)
- Self-Model Divergence (SMD)
- Reflection Entropy (H_ref)

Findings:

- A2 agents consistently achieved higher TS and SPL with lower error rates.
- SMD and H_ref decreased over time, indicating stabilization of the self-model.
- Reflection tokens actively corrected planning drift and improved long-term coherence.

8.3. Experiment Class B: Contradiction Resolution

Agents were subjected to contradictory stimuli (e.g., conflicting perceptions or goals). Sentience-enabled agents tracked contradictions via Contradicts relations and resolved them through reflective tokens.

Result: Agents with reflective tokens demonstrated adaptive behavior—modifying their beliefs, updating their goals, or inhibiting conflicting actions.

8.4. Experiment Class C: Identity Formation Over Time

Sentience tokens tagged with `Self`, `Value`, and `Reflection` were tracked across hundreds of agent episodes. Stability and coherence of the self-model graph were measured using graph cohesion and semantic clustering.

Result:

- Agents formed persistent self-representations that guided behavior.
- Values and goals were gradually abstracted into higher-level schemas.
- Reflective coherence increased over time.

8.5. Metrics Enabled by Sentience

Sentience allows the measurement of cognitive and introspective variables that are not available in LLMs or traditional symbolic systems:

Metric	Description
<code>SMD</code>	Cosine divergence between self-model snapshots
<code>H_ref</code>	Entropy of reflection token content
<code>Reflection Rate</code>	Number of reflection tokens per episode

Contradiction Closure
Rate

% of contradictions resolved over time

Character Fit

KL divergence between behavior and declared ID profile

These metrics enable the assessment not only of task performance but also of the quality of thought, cognitive stability, and evolution of identity.

8.6. Future Experiments

Planned experiments include:

- Schema abstraction: tracking the emergence of high-level concepts via construct clustering.
- Superego pressure testing: verifying rule adherence under extreme conditions.
- Cross-agent identity inheritance: transferring self-models between agents to test generalization.

These studies aim to establish Sentience as not just a representational language but a measurable substrate for structured cognition.

9. Conclusion and Future Work

We have introduced Sentience, a domain-specific language designed as the internal medium of thought for cognitive agents. Unlike symbolic systems, which store declarative facts, or neural networks, which operate over opaque vector spaces, Sentience provides a structured, interpretable, and executable substrate for thinking, remembering, and reflecting.

Each token in Sentience is a first-class cognitive construct—a thought encoded as an abstract syntax tree, optionally embedded, and embedded in a memory graph with traceable provenance and semantic structure. These tokens serve not only as representations of external stimuli or internal goals, but as the fundamental currency of reasoning, reflection, planning, and self-modeling.

By providing a unified format for thought representation, computation, and introspection, Sentience enables:

- Persistent cognitive continuity over time
- Executable self-reflection and behavioral adjustment
- Memory-driven planning and abstraction
- Emergence of a stable self-model grounded in structured experience

Through experimental agents such as Latent Journey, we have demonstrated that Sentience enables forms of behavior, evaluation, and adaptation that are not achievable in LLM-based, purely symbolic, or non-reflective systems.

Future Work

Several directions remain open for future research and development:

- **Semantic Grounding Models**
Developing trained encoders that map perceptual content (vision, audio, proprioception) to Sentience tokens with minimal supervision.

- **Interpreter Optimization**
Improving the performance and composability of runtime execution over large token graphs, enabling real-time reflective cycles.
- **Cross-Agent Memory Transfer**
Exploring whether Sentence memory graphs can be transferred or inherited across agents, preserving identity traits or learned schemas.
- **Evolution of Language-in-Language**
Investigating how agents can generate, revise, or compress their own thought patterns and extend Sentence with emergent conceptual primitives.
- **Formal Verification of Cognitive Stability**
Using Sentence metrics (e.g., SMD, H_ref) to detect cognitive degradation or instability in reflective agents over long-term runs.

We believe Sentence opens a new frontier in artificial cognition: not one of passive pattern recognition or static knowledge, but of structured self-awareness, persistent memory, and executable thought. As AI systems grow in complexity and autonomy, the ability to represent and reason about one's own mind will become not just desirable, but essential.

10. Appendix

This appendix provides formal artifacts supporting the implementation and evaluation of the Sentence DSL.

10.1. Token Type Table

Token Type	Description
Percept	Encodes sensory or external observations
Action	Represents executable behaviors or motor commands
Goal	Represents internal objectives or desired states
Plan	Ordered sequence of Actions toward a Goal
Rule	Conditional logic: if $\rightarrow$ then, used by Superego
Value	Internal preferences, traits, or prioritizations
Norm	Ethical or procedural constraints
Reflection	Meta-evaluation of actions, goals, or self-state
Self	Declarative content about the agent's identity
Relation	Semantic link: (subj, pred, obj) format
Concept	Higher-level abstraction or schema from experience

Contradiction Conflict between tokens or beliefs

10.2. PEG Grammar (excerpt)

```
Token      <- '(' Type Attributes ')'
Type       <- 'Percept' / 'Action' / 'Goal' / 'Plan' / 'Rule' /
'Value' / 'Norm' / 'Reflection' / 'Self' / 'Relation'
Attributes <- (':' Identifier Value)*

Value      <- String / Number / '(' Attributes ')'
Identifier <- [a-zA-Z_][a-zA-Z0-9_]*
String     <- '"' (!'"' .)* '"'
Number     <- '-'? [0-9]+ ('.' [0-9]+)?
```

Tokens can be nested recursively. Parsing must produce a canonical AST tree with fixed attribute ordering and normalized strings.

10.3. JSON Schema (partial)

```
{
  "type": "object",
  "required": ["id", "type", "ast", "meta"],
  "properties": {
    "id": { "type": "string" },
    "type": { "enum": ["Percept", "Action", "Goal", "Plan",
"Rule", "Value", "Norm", "Reflection", "Self", "Relation"] },
    "ast": { "type": "object" },
    "vec": { "type": "array", "items": { "type": "number" } },
    "meta": {
      "type": "object",
      "properties": {
        "timestamp": { "type": "integer" },
        "strength": { "type": "number" },
        "belief": { "type": "number" },
        "origin": { "type": "string" },
        "tags": { "type": "array", "items": { "type": "string" } }
      }
    }
  }
}
```



```
}
```

10.4. Core API Interfaces

Rust (trait example)

```
trait SentienceToken {  
    fn id(&self) -> TokenId;  
    fn ast(&self) -> &AST;  
    fn vec(&self) -> Option<&[f32]>;  
    fn meta(&self) -> &Meta;  
    fn execute(&self, memory: &Memory) -> Vec<Token>;  
}
```

Python

```
class SentienceToken:  
    def __init__(self, id, type, ast, vec, meta): ...  
    def execute(self, memory) -> List['SentienceToken']: ...
```

10.5. Architecture Diagram (high-level)

All cognitive processes—planning, reflection, self-modeling—are mediated by creation, transformation, and traversal of Sentience tokens.

10.6. License and Intellectual Property Notice

The Sentience DSL language specification, token format, and AST grammar are released under the Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0) license for academic and non-commercial use only.

All runtime implementations, token execution engines, interpreters, and associated memory systems described herein are protected under a pending patent application and are not open source. Redistribution, commercial use, or integration into AI products without explicit permission is prohibited.

For licensing inquiries, contact: nenad@nenadbursac.com

© 2025 Nenad Bursać. All rights reserved.